

■ AI ke liye

MATHEMATICS

Zero se Top 1% tak — Hinglish mein

Real AI Problems • Live Data • Algorithms • Practice Questions

LINEAR ALGEBRA |
CALCULUS |
PROBABILITY |
DISCRETE MATH

Designed for Builders — Not Bookworms

■ Contents

PART 1	LINEAR ALGEBRA	Matrix ka duniya — Data ki language	pg 3
PART 2	CALCULUS	Training ka engine — Backprop ki soul	pg 20
PART 3	PROBABILITY & STATISTICS	Uncertainty ko samajhna	pg 35
PART 4	DISCRETE MATH & LOGIC	Computing ki neev	pg 52
BONUS	PRACTICE QUESTIONS	Har topic ke solved problems	In each section

LINEAR ALGEBRA

Data ki Asli Language — Har AI ka Backbone

Yeh kyun zaroori hai?

Socho ek 4K image — usmein 3 crore se zyada numbers hain (pixels). Neural network ko in numbers ko process karna hai. Matrix aur vector ke bina yeh impossible hai. GPT jaisa model? Uske andar sirf matrix multiplications ho rahe hain — kuch aur nahi!

1.1 Vectors — Direction wala Number

Ek **vector** simply ek list of numbers hai — lekin har number ka ek meaning hota hai. Socho tum apni location batana chahte ho — tum sirf ek number nahi doge, tum doge: (latitude, longitude) = (28.6, 77.2). Yeh ek vector hai!

■ Real AI Example: Word Embedding

ChatGPT mein har word ek vector ban jaata hai. Jaise: 'King' = [0.8, 0.3, -0.5, 0.9, ...] (512 numbers) 'Queen' = [0.7, 0.4, -0.4, 0.8, ...] (512 numbers) In do vectors ka difference nearly same hai jaise: 'Man' aur 'Woman' ka difference — isliye AI ko pata chalta hai ke King - Man + Woman = Queen!

Vector Operations — Kaise kaam karte hain:

1. Addition: Do vectors add karo — corresponding numbers add hote hain.

$$\mathbf{a} = [1, 2, 3], \mathbf{b} = [4, 5, 6] \Rightarrow \mathbf{a} + \mathbf{b} = [5, 7, 9]$$

AI Use: Jab hum do word embeddings add karte hain, toh hum unke meanings combine kar rahe hote hain.

2. Dot Product (Most Important!): Kitne similar hain do vectors?

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}[0]*\mathbf{b}[0] + \mathbf{a}[1]*\mathbf{b}[1] + \mathbf{a}[2]*\mathbf{b}[2] = 1*4 + 2*5 + 3*6 = 32$$

AI Use: Attention mechanism mein (GPT, BERT sab mein!) — dot product se pata chalta hai ke kaunsa word kaunse word par 'dhyan' de. Bada dot product = zyada related words!

3. Magnitude (Length): Vector kitna bada hai?

$$|a| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14} \approx 3.74$$

■ Vector Visualization (2D mein):

Y-axis | 3 | * (2,3) 2 | / 1 | / 0 | --/----- X-axis 0 1 2 Yeh vector [2, 3] hai
— matlab 'X mein 2 step, Y mein 3 step jao' Length = $\sqrt{4+9} = \sqrt{13} \approx 3.6$

==■ Practice Questions — Vectors

Q1. $a = [3, 4]$ ka magnitude kya hoga?

■ Sol: $\sqrt{3^2 + 4^2} = \sqrt{9+16} = \sqrt{25} = 5$. Yeh classic 3-4-5 triangle hai!

Q2. $a = [1,0,1]$ aur $b = [1,1,0]$ ka dot product nikalo.

■ Sol: $1*1 + 0*1 + 1*0 = 1$. Matlab ye partially similar hain.

Q3. Agar 'Delhi' = $[0.5, 0.8]$ aur 'Mumbai' = $[0.6, 0.7]$ hain toh dot product batao aur explain karo.

■ Sol: $0.5*0.6 + 0.8*0.7 = 0.30 + 0.56 = 0.86$. High value matlab dono vectors similar direction mein hain — AI ko lagega ye dono related concepts hain!

1.2 Matrices — Vectors ka Superhero Version

Ek matrix simply vectors ka group hai — rows aur columns mein arranged. Ek 3x3 matrix matlab 9 numbers, ek 1000x768 matrix matlab 7,68,000 numbers. GPU yahi karta hai — billions of matrix operations per second!

■ Real AI Example: Image Processing

Ek 28x28 pixel grayscale image (jaise MNIST digit): [[0, 0, 255, 200, ...], <-- Row 1: top pixels
[0, 150, 255, 180, ...], <-- Row 2 ...28 rows total...] Yeh ek 28x28 matrix hai. Neural network isko process karne ke liye iske saath weight matrix multiply karta hai. Bas itna hi hai!

Matrix Multiplication — Neural Network ka Core:

Maan lo input hai X (data) aur weights hain W. Neural network ka ek layer basically: **Output = X × W**

$$X = \begin{bmatrix} 1 & 2 \end{bmatrix}, W = \begin{bmatrix} 5 & 6 \end{bmatrix}, X \cdot W = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \end{bmatrix}$$

GPU kyun zaroori hai?

Matrix multiplication bahut repetitive hai — same operation crores baar. CPU ek baar mein 1-2 operations karta hai. GPU ek baar mein 10,000+ operations parallel karta hai. Isliye RTX 4090 pe ChatGPT ka response 1 second mein aata hai, CPU pe ghanton lagenge!

Transpose — Matrix ko Rotate karna:

$$A = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}, A^T = \begin{bmatrix} 1 & 4 & 5 & 6 \end{bmatrix}$$

AI Use: Backpropagation mein gradients calculate karte waqt transpose chahiye hota hai.

Inverse — Matrix ka Ulta:

Agar A ek matrix hai, toh A^{-1} woh matrix hai jiska A ke saath multiply karne par Identity matrix milti hai. Yeh solving equations mein use hoti hai — jaise Linear Regression ka exact solution.

$$A \times A^{-1} = I \text{ (Identity Matrix — diagonal mein 1, baaki 0)}$$

Determinant — Matrix ki 'Value':

2x2 matrix ke liye: $\det(\begin{bmatrix} a & b \\ c & d \end{bmatrix}) = a \cdot d - b \cdot c$

AI Use: Agar determinant zero hai toh matrix singular hai — matlab data mein redundant information hai aur inverse exist nahi karta. PCA mein yeh problem detect karne ke liye use hota hai.

— ■ Practice Questions — Matrices

Q1. $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ ka determinant nikalo.

■ Sol: $\det = 2 \cdot 3 - 1 \cdot 1 = 6 - 1 = 5$. Non-zero hai toh matrix invertible hai.

Q2. Ek neural network layer: Input = $[1, 2, 3]$ (1x3), Weights = $\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ (3x2). Output kya hoga?

■ Sol: Output = $[1 \cdot 1 + 2 \cdot 0 + 3 \cdot 1, 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 1] = [4, 5]$. Shape: 1x2. Input ke 3 features se 2 output features ban gaye!

Q3. Ek color image (RGB) 64x64 pixels hai toh uska matrix shape kya hoga?

■ Sol: $64 \times 64 \times 3$ = ek 3D tensor. 3 channels (Red, Green, Blue) mein se har ek 64x64 matrix hai. Total 12,288 numbers!

1.3 Eigenvalues & Eigenvectors — Matrix ki Hidden Power

Yeh thoda abstract lagta hai — lekin ek simple idea hai. Jab tum ek matrix ko kisi vector pe apply karte ho, toh vector usually rotate bhi hota hai aur scale bhi. **Eigenvector** woh special vector hai jo sirf scale hota hai — rotate nahi! Aur scale factor ko **eigenvalue** kehte hain.

$$A \times v = \lambda \times v \quad (A = \text{matrix}, v = \text{eigenvector}, \lambda = \text{eigenvalue})$$

■ Real AI Example: PCA (Principal Component Analysis)

Socho tumhare paas 1000 logo ka data hai jismein 50 features hain (height, weight, income, age...). Bahut zyada dimensions! PCA kya karta hai: 1. Data ka covariance matrix banao 2. Uske eigenvectors nikalo 3. Top eigenvectors (jinka eigenvalue bada ho) woh 'important directions' hain 4. In directions pe project karo — 50 features se sirf 2-3 bana do! Result: 50D data ko 2D mein plot kar sakte ho bina zyada information lose kiye. Face recognition mein yahi 'Eigenfaces' technique hai!

■ Eigenvalue Intuition:

```
Bada Eigenvalue => Is direction mein data zyada spread hai => Important! Chhota Eigenvalue => Is direction mein data kam spread hai => Less important Google PageRank bhi eigenvector use karta hai! Web pages ka graph banao -> transition matrix banao -> uska dominant eigenvector = page importance scores
```

Q1. Agar $A = \begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$ hai toh eigenvalues kya hain?

■ Sol: Yeh diagonal matrix hai — eigenvalues directly diagonal elements hain: $\lambda_1=3$, $\lambda_2=5$. Eigenvectors $[1,0]$ aur $[0,1]$ hain.

Q2. PCA mein agar top 2 eigenvalues hain $[120, 80, 5, 2, 1, \dots]$, toh hum kitne components rakhein?

■ Sol: Top 2 eigenvalues ($120+80=200$) total $200/208 = 96\%$ variance explain karte hain. 2 components kaafi hain!

1.4 SVD — Singular Value Decomposition (Most Powerful!)

SVD kisi bhi matrix ko teen matrices mein tod deta hai. Yeh AI mein sabse zyada use hone wali techniques mein se ek hai — recommendations se leke LLM compression tak.

$$A = U \times S \times V^T \text{ (koi bhi matrix } A = \text{teen matrices ka product)}$$

U: Left singular vectors – input patterns **S:** Diagonal matrix of singular values – importance weights **V^T:** Right singular vectors – output patterns

■ Real AI Example: Netflix Recommendation

Users x Movies ratings matrix socho (10M users x 500K movies = huge!): User1: [5, ?, 4, 1, ?, 5, ...] User2: [?, 3, ?, 5, 4, ?, ...] (? = unrated movies) SVD karta hai: 1. Matrix decompose karo $U \times S \times V^T$ mein 2. Sirf top-k singular values rakho (baaki drop karo) 3. Matrix reconstruct karo — '?' values fill ho jaate hain! Yeh 'Collaborative Filtering' hai. Netflix \$1M prize SVD se jita tha!

■ Real AI Example: LLM Compression (LoRA)

GPT-4 mein billions of parameters hain. Fine-tuning expensive hai. LoRA (Low-Rank Adaptation) kya karta hai: - Weight matrix W (bada) = U (tall) x V (wide) - SVD se low-rank approximation lo - Sirf U aur V ko train karo (much fewer params!) Result: 10x kam parameters train karo, almost same accuracy pao. Isliye tum apna custom GPT fine-tune kar sakte ho!

Q1. SVD mein agar $S = [100, 80, 5, 2, 0.1]$ hai toh top-2 approximation kitna variance capture karta hai?

■ Sol: Total = $100+80+5+2+0.1 = 187.1$. Top-2 = 180. $180/187.1 = 96.2\%$ variance. Excellent compression!

Q2. Ek 1000×500 matrix hai. SVD ke baad hum rank-10 approximation lete hain. Kitne numbers save karte hain?

■ Sol: Original: $1000 \times 500 = 500,000$ numbers. Rank-10: $U(1000 \times 10) + S(10) + V^T(10 \times 500) = 10000+10+5000 = 15,010$. 97% compression!

1.5 Spaces & Transformations — Data ka Journey

Jab data neural network ke through jaata hai, toh har layer ek **linear transformation** apply karti hai. Matrix multiplication = data ko ek space se doosre mein le jaana.

Neural Network ka Data Flow:

```
Input Layer: 784 numbers (28x28 image flattened) | W1 (784x256) matrix multiply  
Hidden Layer: 256 numbers (new representation) | W2 (256x128) matrix multiply Hidden  
Layer: 128 numbers | W3 (128x10) matrix multiply Output Layer: 10 numbers  
(probabilities for digits 0-9) Har step ek matrix multiplication hai! Data literally  
ek 'space' se doosre mein transform hota hai.
```

Column Space & Null Space — Kya hota hai?

Column Space: Woh saare points jahan matrix pahuncha sakta hai. Neural network ka column space = possible outputs ka set. Null Space: Woh inputs jinha matrix zero mein bhejta hai = information jo network ignore karta hai. High-dimensional data processing mein yeh samajhna zaruri hai ki kya learn ho sakta hai aur kya nahi!

CALCULUS

Training ka Engine — Model kaise Seekhta hai?

Ek Line mein Calculus:

Tumhari model galti karti hai. Calculus batata hai: 'Iss galti ko kam karne ke liye kis direction mein weights change karo.' Bas itna. Derivative = 'change ka rate' — aur yahi learning hai!

2.1 Derivatives — Change ka Measure

Derivative batata hai: 'Agar input mein thoda change aaye, toh output mein kitna change aayega?' Yeh slope hai — function kitna steep hai us point pe.

$$dy/dx = \lim_{\delta \rightarrow 0} [f(x+\delta) - f(x)] / \delta$$

Simple Rules (yeh yaad kar lo!):

$$d/dx (x^n) = n * x^{(n-1)}$$

$$d/dx (x^2) = 2x \mid d/dx (x^3) = 3x^2 \mid d/dx (5) = 0$$

$$d/dx (e^x) = e^x \text{ (exponential apna derivative khud hai!)}$$

$$d/dx (\log x) = 1/x$$

■ Real AI Example: Loss Function Minimize karna

Maan lo Loss $L = (y_{\text{pred}} - y_{\text{true}})^2$ (Mean Squared Error) $L = (w*x - y)^2$ jahan w = weight, x = input, y = true value $dL/dw = 2*(w*x - y) * x$ [chain rule use hua!] Yeh batata hai: 'w mein thoda change karo, Loss mein kitna change aayega' Negative direction mein jao => Loss kam hogi => Model better hoga!

Chain Rule — Backpropagation ka Heart:

Agar $y = f(g(x))$ hai, toh $dy/dx = f'(g(x)) * g'(x)$. Matlab: Agar ek function ke andar doosra function hai, toh derivatives multiply karo.

$$dL/dw_1 = dL/dL_{\text{out}} \times dL_{\text{out}}/dz_2 \times dz_2/dz_1 \times dz_1/dw_1$$

AI Use: Backpropagation exactly yahi karta hai — output se leke pehle layer tak, chain rule se har weight ka gradient calculate karta hai!

Q1. $f(x) = 3x^3 + 2x^2 - 5x + 10$ ka derivative nikalo.

■ Sol: $f'(x) = 9x^2 + 4x - 5$. Rule: $nx^{(n-1)}$ use kiya har term pe.

Q2. Loss = $(w - 3)^2$ hai. dL/dw nikalo aur bolo $w=5$ pe gradient kya hai?

■ Sol: $dL/dw = 2(w-3)$. $w=5$ pe: $2*(5-3) = 4$. Matlab w ko 4 units ka push hai positive direction mein — isliye hum w kam karenge!

Q3. $y = \text{sigmoid}(wx + b) = 1/(1+e^{-(wx+b)})$. dL/dw chain rule se kaise nikalenge?

■ Sol: $dL/dw = dL/dy * dy/d\sigma * d\sigma/dz * dz/dw$. Yahan $dz/dw = x$, $d\sigma/dz = \sigma*(1-\sigma)$. So $dL/dw = dL/dy * \sigma*(1-\sigma) * x$. Yahi backprop hai!

2.2 Partial Derivatives — Multi-Variable ka Game

Neural network mein millions of weights hain. Partial derivative batati hai: 'Sirf EK weight change karo, baaki sab fixed rakho — Loss mein kitna change aayega?'

$\partial L / \partial w_i$ = Loss ko sirf w_i ke respect mein differentiate karo

■ Real AI Example: Weight Update in Training

Ek simple neural net: $L = (w_1 \cdot x_1 + w_2 \cdot x_2 - y)^2$
 $\partial L / \partial w_1 = 2 \cdot (w_1 \cdot x_1 + w_2 \cdot x_2 - y) \cdot x_1$
 $\partial L / \partial w_2 = 2 \cdot (w_1 \cdot x_1 + w_2 \cdot x_2 - y) \cdot x_2$
Dono weights simultaneously update hote hain: $w_{1_new} = w_1 - lr \cdot (\partial L / \partial w_1)$
 $w_{2_new} = w_2 - lr \cdot (\partial L / \partial w_2)$
Yeh gradient descent ka ek step hai! GPT-4 mein yeh 1 trillion weights ke liye hota hai!

2.3 Gradients — Steepest Descent ka Compass

Gradient ek vector hai jo saare partial derivatives combine karta hai. Yeh vector point karta hai: 'Is direction mein function sabse fast increase karta hai.' ML mein hum opposite direction mein jaate hain — Loss minimize karne ke liye!

$$\nabla L = [\partial L / \partial w_1, \partial L / \partial w_2, \partial L / \partial w_3, \dots, \partial L / \partial w_n]$$

■ Gradient Descent Analogy:

Socho tum andheri raat mein ek pahad pe ho. Tumhara goal: valley (minimum) mein pahunchna. Strategy: Har step pe pairon ke neeche slope check karo. Sabse steep slope ke ULTI direction mein chalo. Chote-chote kadam (learning rate) lo. Repeat until flat ground (minimum) mile. Gradient = slope ka direction Learning Rate = step ka size Minimum = best model weights!

2.4 Optimization — Training ki Poori Story

Gradient Descent Algorithm:

1. Random weights se start karo
2. Data ek batch pass karo (forward pass)
3. Loss calculate karo
4. Gradients calculate karo (backward pass / backprop)
5. Weights update karo: $w = w - lr \times \text{gradient}$
6. Steps 2-5 repeat karo jab tak loss stable na ho jaaye

Learning Rate — Kitna Important hai?

Too High Learning Rate:

Too Low Learning Rate:

Loss bounce karta hai. Model diverge ho jaata Training bahut slow. Local minimum mein hai. Training fail. phans jaate hain. Days lagna.

■ Real AI: Adam Optimizer (GPT mein yahi use hota hai)

Simple gradient descent slow hai. Adam (Adaptive Moment Estimation) better hai: 1. Momentum: Past gradients bhi consider karo (inertia jaisa) 2. RMSprop: Frequently updated weights ko smaller steps do 3. Result: Automatically learning rate adjust hoti hai per weight! Formula: $w = w - (lr / \sqrt{v+eps}) * m$ m = exponential moving average of gradients v = exponential moving average of squared gradients Typical $lr = 0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$

Q1. $w=5$, $\text{gradient}=2$, $\text{learning_rate}=0.1$ hai. New weight kya hoga?

■ Sol: $w_{\text{new}} = 5 - 0.1*2 = 5 - 0.2 = 4.8$. Gradient positive tha toh w thoda kam hua — Loss minimum ki taraf gaye!

Q2. Agar learning rate 10 kar dein toh kya hoga?

■ Sol: $w_{\text{new}} = 5 - 10*2 = -15$. Bahut bada jump! Agle step pe gradient opposite direction mein bada hoga. Loss bounce karega, model sikh nahi paayega.

Q3. Local vs Global Minimum — kya difference hai AI mein?

■ Sol: Local minimum = ek chhoti valley. Global minimum = sabse deep valley. Simple loss functions mein mostly ek hi minimum hota hai. Deep networks mein bahut saare local minima hain — lekin Adam jaisi optimizers aksar kaafi accha solution dhoondh leti hain!

PROBABILITY & STATISTICS

Uncertainty ko Samajhna — AI ki Sixth Sense

Kyun zaroori hai?

AI kabhi '100% sure' nahi hota — aur hona bhi nahi chahiye! ChatGPT 'probably' sahi answer deta hai. Medical AI '87% probability' cancer detect karta hai. Probability is language of uncertainty — aur uncertainty is reality!

3.1 Probability Basics — Uncertainty ka Number

Probability ek number 0 aur 1 ke beech hai: 0 = Impossible | 0.5 = 50-50 chance | 1 = Certain

Key Concepts:

Independent Events: Ek event doosre ko affect nahi karta. Coin toss ka result pichle toss se independent hai.

$$P(A \text{ and } B) = P(A) \times P(B) \text{ [agar independent]}$$

Conditional Probability: Ek event ka probability, given ke doosra hua hai.

$$P(A|B) = P(A \text{ and } B) / P(B) \text{ [B ke baad A ka probability]}$$

■ Real AI Example: Spam Filter

Email classification: $P(\text{spam}) = 0.3$ (30% emails spam hain) $P(\text{word 'lottery' | spam}) = 0.7$ (spam mein 70% baar 'lottery' aata hai) $P(\text{word 'lottery' | not_spam}) = 0.01$ (normal email mein 1%) Email mein 'lottery' dikha — spam hai ya nahi? => Bayes Theorem use karo! (Next section mein)

3.2 Probability Distributions — Data ke Shapes

Distribution	Shape/Use	AI Application
Normal (Gaussian)	Bell curve — symmetric	Weight initialization, noise modeling, data augmentation

Bernoulli	0 ya 1 — coin toss	Binary classification (spam/not-spam, cancer/no-cancer)
Categorical	Multiple classes	Multiclass classification — GPT ka next token prediction
Uniform	Sab equally likely	Random initialization, exploration in RL
Poisson	Count of rare events	Fraud detection — kitne unusual transactions per hour
Exponential	Time between events	Server request timing, failure prediction

■ Real AI Example: Normal Distribution in Neural Nets

Weight initialization: Weights zero se initialize karo — sab same hain, network learn nahi karta!
 Solution: Normal distribution se sample karo: $w \sim N(0, \sqrt{2/n_{\text{inputs}}})$ [He initialization] Kyun?
 Normal distribution ensure karta hai: - Kuch weights positive, kuch negative (symmetry) - Zyada extreme values nahi (bounded by std dev) - Gradients vanishing/exploding nahi hote
 Xavier init, He init — sab normal distribution se!

3.3 Bayes Theorem — AI ka Most Important Formula

Bayes Theorem batata hai: 'Evidence dekho, beliefs update karo.' Yeh ek 'rational mind' ka mathematical model hai.

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

P(A): Prior — pehle se kya pata tha **P(B|A):** Likelihood — A hone pe B kitna likely hai
P(B): Evidence — B ka total probability **P(A|B):** Posterior — B dekhne ke baad A ka updated probability

■ Real AI Example: Medical Diagnosis (Kisi ki life bach sakti hai!)

COVID test ki accuracy: $P(\text{positive} | \text{has_covid}) = 0.99$ (test 99% sensitive) $P(\text{positive} | \text{no_covid}) = 0.05$ (5% false positive) $P(\text{has_covid}) = 0.01$ (1% population infected) Test positive aaya — kya patient ko COVID hai? $P(B) = 0.99 \times 0.01 + 0.05 \times 0.99 = 0.0099 + 0.0495 = 0.0594$
 $P(\text{covid} | \text{positive}) = (0.99 \times 0.01) / 0.0594 = 0.0099 / 0.0594 \approx 0.167$ Sirf 16.7%! Positive test ke baad bhi! Isliye doctors multiple tests karte hain. Bayes ye batata hai — base rate (1%) ignore nahi kar sakte!

■ Real AI Example: Spam Filter (Naive Bayes Classifier)

$P(\text{spam} | \text{words in email}) = P(\text{words} | \text{spam}) \times P(\text{spam}) / P(\text{words})$ Naive Bayes assume karta hai words independent hain: $P('lottery', 'free', 'win' | \text{spam}) = 0.7 \times 0.8 \times 0.9 = 0.504$
 $P('lottery', 'free', 'win' | \text{ham}) = 0.01 \times 0.05 \times 0.02 = 0.00001$ Spam posterior bahut zyada hai => Email spam mark karo! Yeh 1990s ka state-of-art spam filter tha. Aaj bhi Gmail mein use hota hai!

Q1. Ek factory mein 2% products defective hain. Quality test 95% accurate hai. Test ne defective bataya — actually defective hone ki probability?

■ Sol: $P(\text{def}|+) = P(+|\text{def}) \cdot P(\text{def}) / P(+)$ $= (0.95 \cdot 0.02) / (0.95 \cdot 0.02 + 0.05 \cdot 0.98) = 0.019 / 0.068 \approx 0.279$. Sirf 28%! Isliye ek test kaafi nahi hota.

Q2. Naive Bayes mein 'naive' kyun kehte hain?

■ Sol: Kyunki yeh 'naive' assumption karta hai ke sab features independent hain. Reality mein ye nahi hote — 'free' aur 'money' correlated hain spam mein. Par fir bhi yeh bahut well work karta hai aur fast hai!

3.4 Expectation & Variance — Data ka Summary

Mean (Average) / Expectation:

$$E[X] = (x_1 + x_2 + \dots + x_n) / n$$

AI Use: Batch normalization mein mean subtract karte hain — taaki data zero-centered ho aur training stable ho.

Variance — Kitna Spread hai Data:

$$\text{Var}(X) = E[(X - \text{mean})^2] = \text{Average of squared distances from mean}$$

AI Use: High variance = model overfitting kar raha hai (training pe bahut achha, test pe bura). Low variance with high bias = underfitting.

Standard Deviation = $\sqrt{\text{Variance}}$:

■ Real AI Example: Batch Normalization

Deep network train karte waqt, activations drift karte hain (covariate shift). Batch Norm kya karta hai: $x_{\text{norm}} = (x - \text{mean}) / \text{std_dev}$ 1. Mean subtract karo => zero centered 2. Std dev se divide karo => unit variance Result: Training 10x fast hoti hai! Yeh ResNet, BERT, GPT sab use karte hain. Mean aur variance real-time calculate hoti hai is batch ke liye.

3.5 Maximum Likelihood Estimation (MLE) — Model Learning

MLE ka idea: 'Woh parameters dhoondho jo observed data ko most likely banayein.' Training = MLE!

$$\theta^* = \text{argmax } P(\text{data} | \theta) \Rightarrow \text{Woh theta jiske saath data most likely ho}$$

■ Real AI Example: Training a Language Model

Training data: 'The cat sat on the mat' MLE kya karta hai: $P(\text{data} | \theta) = P(\text{'The'} | \theta) \times P(\text{'cat'} | \text{'The'}, \theta) \times P(\text{'sat'} | \text{'The cat'}, \theta) \times \dots$ Maximize karo yeh probability => θ seekh jaata hai! In practice: log-likelihood maximize karte hain (log monotonic hai) $\log P(\text{data} | \theta) = \text{sum of log probabilities}$ Yeh exactly cross-entropy loss minimize karne ke barabar hai — isliye GPT cross-entropy loss use karta hai!

3.6 Information Theory — LLM ka Secret Sauce

Entropy measure karta hai: 'Kitni uncertainty hai?' High entropy = high uncertainty = more information.

$$H(X) = - \sum P(x) \times \log(P(x))$$

Cross-Entropy Loss — Har Classification Model mein:

$$CE = - \sum y_true \times \log(y_pred) \text{ [} y_true = \text{actual, } y_pred = \text{model output]}$$

AI Use: GPT, BERT, every classifier minimize karta hai cross-entropy. Agar model perfect predict kare toh CE = 0. Wrong predictions pe CE large hoti hai.

KL Divergence — Do Distributions ka Difference:

$$KL(P||Q) = \sum P(x) \times \log(P(x)/Q(x))$$

■ Real AI Example: Variational Autoencoders (VAE) & RLHF

VAE (image generation): - Latent space ko Normal distribution ke close rakhna hai - Loss = Reconstruction Loss + $KL(q(z|x) || N(0,1))$ - KL term punish karta hai agar latent distribution normal se door ho
RLHF (ChatGPT training): - Fine-tuned model bahut base model se door na ho jaaye - KL penalty add karte hain: Loss += beta * $KL(\text{fine_tuned} || \text{base})$ - Isliye ChatGPT helpful bhi hai aur 'sane' bhi rehta hai!

Q1. Ek fair coin ka entropy kya hai?

■ Sol: $H = -(0.5 \times \log_2(0.5) + 0.5 \times \log_2(0.5)) = -(0.5 \times (-1) + 0.5 \times (-1)) = 1$ bit. Maximum entropy kyunki maximum uncertainty hai!

Q2. Model ne predict kiya: $P(\text{cat})=0.7$, $P(\text{dog})=0.2$, $P(\text{bird})=0.1$. Actual answer 'cat' hai.

Cross-entropy loss kya hai?

■ Sol: $CE = -(1 \times \log(0.7) + 0 \times \log(0.2) + 0 \times \log(0.1)) = -\log(0.7) \approx 0.357$. Agar model 100% confident hota toh $CE = -\log(1) = 0$ hota!

DISCRETE MATHEMATICS & LOGIC

Computing ki Neev — Code ke peeche ka Math

Kyun zaroori hai?

Linear algebra, calculus, probability — yeh sab continuous math hai. Lekin computers discrete hain — 0s aur 1s. Graphs, algorithms, complexity — sab discrete math se aate hain. Knowledge graphs, search algorithms, AI planning — sab yahan hai!

4.1 Boolean Logic & Logic Gates — Computer ka DNA

Boolean algebra mein sirf do values hain: True (1) ya False (0). Har ek GPU operation ultimately in logic gates pe run hoti hai!

Operation	Symbol	Rule	AI Use
AND	$A \wedge B$	Both true $\Rightarrow$ true	Agar spam AND malicious $\Rightarrow$ block
OR	$A \vee B$	At least one true $\Rightarrow$ true	Agar rain OR cold $\Rightarrow$ wear jacket
NOT	$\neg A$	Flip: true $\Rightarrow$ false	Complement of prediction
NAND	$\neg(A \wedge B)$	AND ka opposite	Universal gate — CPUs build karte hain
XOR	$A \oplus B$	Different $\Rightarrow$ true	Error detection, encryption

■ Real AI Example: Neural Network Activation as Logic

$\text{ReLU}(x) = \max(0, x)$ --> Basically ek 'soft' logic gate hai! Agar input positive hai (information active hai) $\Rightarrow$ pass through Agar input negative hai (not relevant) $\Rightarrow$ block (zero) Deep network = millions of such logical decisions stacked together! GELU, Sigmoid — sab smooth approximations of logic gates hain.

4.2 Graph Theory — Connections ka Math

Graph = Nodes (objects) + Edges (connections). AI mein everywhere graphs hain — social networks, knowledge bases, molecules, circuits.

Graph $G = (V, E)$ V = vertices (nodes), E = edges (connections)

■ Real AI Examples: Graphs in Action

1. Knowledge Graph (Google, ChatGPT plugins): Node: 'Delhi' --[capital_of]--> Node: 'India' Node: 'India' --[has_currency]--> Node: 'Rupee' AI can reason: Delhi ka currency = Rupee! 2. Social Network Graph (Facebook AI): Users = nodes, friendships = edges Friend recommendation = 'friends of friends with similar edges' 3. Molecule Graph (Drug Discovery AI): Atoms = nodes, chemical bonds = edges Graph Neural Networks predict drug properties! 4. Transformer Attention = Fully Connected Graph: Every word = node, attention weight = edge weight 'Woh aadmi jo kal aaya tha...' => 'woh' aur 'aadmi' ka edge strong!

BFS & DFS — Search Algorithms:

BFS (Breadth-First Search): Level by level explore karo — shortest path dhoondho.

DFS (Depth-First Search): Ek path pe jaate raho — exhaustive search.

■ Real AI: A* Search in AI Planning

Game AI (chess engine, navigation): BFS: Har possible move explore karo level by level A*: BFS + heuristic (estimated distance to goal) $f(n) = g(n) + h(n)$ $g(n)$ = actual cost so far $h(n)$ = estimated cost to goal (heuristic) Google Maps route finding = A* on road graph AlphaGo = MCTS (Monte Carlo Tree Search) on game graph

4.3 Combinatorics — Counting ka Math

Permutations: Order matters — n items mein se r choose karo (order matters)

Combinations: Order doesn't matter — n items mein se r choose karo (order nahi)

$$P(n,r) = n! / (n-r)! \quad C(n,r) = n! / (r! \times (n-r)!)$$

■ Real AI Example: Hyperparameter Search

Tumhare paas 3 hyperparameters hain: Learning rate: [0.1, 0.01, 0.001] (3 options) Batch size: [32, 64, 128] (3 options) Layers: [2, 3, 4] (3 options) Total combinations = $3 \times 3 \times 3 = 27$ 5 hyperparameters each with 10 options = $10^5 = 100,000$ combinations! Random Search (combinations se sample karo) vs Grid Search (sab try karo) Bayesian Optimization = intelligent combinations explore karna!

4.4 Big-O Notation — Algorithm ki Efficiency

Big-O batata hai: 'Agar input size n baar bade, time/memory kitni baar badhega?' Yeh non-negotiable hai — ek slow algorithm real AI systems mein useless hai.

Notation	Name	Example	Scaling ($n=1M$)
$O(1)$	Constant	Hash lookup, array index	1 op
$O(\log n)$	Logarithmic	Binary search, BST	20 ops
$O(n)$	Linear	Loop through array, BFS	1M ops
$O(n \log n)$	Linearithmic	Merge sort, FFT	20M ops
$O(n^2)$	Quadratic	Nested loops, naive attention	1 Trillion ops
$O(2^n)$	Exponential	Brute force subset problems	NEVER!

■ Real AI Critical Example: Transformer Attention Complexity

Original Transformer: $O(n^2)$ attention n = sequence length (tokens) $n=1000$: 1,000,000 operations $n=100,000$: 10,000,000,000 operations => SLOW! Flashattention: $O(n)$ approximate — 10x faster! Mamba (State Space Models): $O(n)$ exact — future of LLMs? Isi liye GPT-4 ka context window limit hai — $O(n^2)$ expensive hai. Researchers Big-O improve karte hain => better AI!

Q1. Sorted array mein binary search $O(\log n)$ hai. $n=1024$ pe max kitne steps?

■ Sol: $\log_2(1024) = 10$ steps. Direct comparison se 1024 steps. Binary search 100x fast!

Q2. Naive matrix multiplication $O(n^3)$ hai. $n=1000$ mein kitne operations?

■ Sol: $1000^3 = 1$ billion ops. Strassen algorithm $O(n^{2.807})$ karta hai = ~630M ops. Modern AI uses BLAS routines optimized for GPUs!

Q3. Ek hash map $O(1)$ lookup deta hai. 1M users ke beech user dhundhne mein normal array vs hash map mein difference?

■ Sol: Array: $O(n) = 1M$ comparisons average. Hash Map: $O(1) = 1$ operation. Real-time recommendation systems hash maps use karte hain!

■ MASTER CHEAT SHEET

Saare formulas ek jagah — Print karke wall pe lagao!

Concept	Formula / Key Idea	AI Application
Dot Product	$a \cdot b = \sum a_i \cdot b_i$	Attention, Cosine Similarity
Matrix Mult	$(AB)_{ij} = \sum A_{ik} \cdot B_{kj}$	Neural Network Layers
Eigenvalue	$Av = \lambda v$	PCA, PageRank
SVD	$A = U \Sigma V^T$	Recommendations, LoRA
Derivative	$dy/dx = \lim [f(x+h) - f(x)]/h$	Gradient Calculation
Chain Rule	$dz/dx = dz/dy \cdot dy/dx$	Backpropagation
Gradient Descent	$w = w - lr \cdot \nabla L$	Training Any Model
Bayes Theorem	$P(A B) = P(B A) \cdot P(A) / P(B)$	Probabilistic AI, Spam
Cross-Entropy	$CE = -\sum y \cdot \log(\hat{y})$	Classification Loss
KL Divergence	$KL(P Q) = \sum P \cdot \log(P/Q)$	VAE, RLHF, GANs
Entropy	$H = -\sum p \cdot \log_2(p)$	Information Theory
Softmax	$p_i = e^{x_i} / \sum e^{x_j}$	Final layer probabilities
Big-O	$O(1) < O(\log n) < O(n) < O(n^2)$	Algorithm Efficiency
MLE	$\theta^* = \operatorname{argmax} P(\text{data} \theta)$	Model Parameter Learning

■ Ab Aage Kya Karo?

1. Ye PDF ek baar poori padho — concepts samjho, formulas ratna mat. 2. Python mein implement karo: numpy ke saath matrices, scipy ke saath SVD, matplotlib ke saath gradients plot karo. 3. Ek scratch neural network banao: forward pass + backprop sirf numpy se. 4. Kaggle pe competitions join karo — real data pe yeh math use karo. 5. Papers padho: 'Attention Is All You Need', 'Deep Residual Learning' — in papers mein yahi math hai jo tumne abhi seekha.

Yaad rakho: Top 1% engineers woh nahi hote jo sab formulas yaad karte hain. Woh hote hain jo UNDERSTAND karte hain — aur apply karna jaante hain.

Best of luck! You've got this. ■